

Palantir Vertex 深度技术调研报告：面向本地复刻“基于本体的场景模拟与数字孪生推演引擎”

撰写日期：2026 年 5 月。本文基于 Palantir 官方文档（palantir.com/docs/foundry/vertex/、palantir.com/docs/foundry/workshop/scenarios-、palantir.com/docs/foundry/functions/、palantir.com/docs/foundry/foundry-branching/）、官方白皮书（[Vertex for Oil & Gas](#)）、Foundry 公开专利、社区与第三方分析文章。对未公开实现细节均明确标注【推测】。

一、产品定位与核心概念

1.1 在 Foundry 产品矩阵中的位置

Vertex 在 Foundry 中归类为 **Operational Application（操作型应用）**，与 Object Explorer、Workshop、Quiver、Map 并列。它不是一个独立的数据建模或编排工具，而是**消费层**——直接坐落在已构建好的 Ontology 之上，负责把“对象 + 链接 + 模型 + 时间序列 + Action”组装成一个可交互的“系统图（System Graph）”用于决策模拟（[Vertex Configuration docs](#)）。

与其他 Foundry 应用的关系总结：

应用	角色	与 Vertex 的关系
Pipeline Builder	ETL / 数据集构建	上游：产生支撑对象类型的数据集；Vertex 不直接使用，但其结果通过 Ontology 索引才能进入 Vertex
Ontology Manager	对象/链接/Action 类型定义；type-class 元数据	上游：Vertex 完全消费 Ontology Manager 中声明的对象类型、链接类型、time series property、Action type、Vertex 专用 type-classes（如 <code>vertex:link_primary_direction</code> ）
Modeling Objectives	ML 模型版本与部署管理	上游：Vertex 引用 Objective 绑定到 Ontology 的模型；legacy 模型选择路径已 sunset，推荐方式是 Function on Model → 函数式 Action
Functions (TypeScript v1/v2、	业务逻辑/模型封装/Search Around 函数/Derived Property	双向：Vertex 调用 Functions 来 (a) 包装模型推理 (b) 生成自定义图（Search Around Function）(c) 计算节点上的派生属性

Python)	函数	
Workshop	通用低代码应用构建器，自身亦有 Scenarios 能力	平行 + 嵌入：Vertex 图可作为 widget 嵌入 Workshop 模块 ；Vertex 的 Scenario 与 Workshop 的 Scenario 共享相同底层 Ontology fork / Scenario 机制 （详见第八章）
Contour	表格式分析（已逐步被 Object Explorer/Quiver 替代）	弱关系；用于探索数据集，但不进入 Vertex
Quiver	交互式数据分析/时间序列详查	协作：Vertex 选中节点上的时间序列可右键 “Open in Quiver” 做更细分析
Object Explorer	对象搜索与浏览	协作：Vertex 中的时间序列可“Open in Object Explorer”；URL 参数 <code>objectRid</code> 、 <code>objectSetRid</code> 也用来把 Object Explorer 选中的对象集预填进 Vertex（ generate graph from apps ）
Map	地理空间可视化	平行：同样消费 Ontology，但视角是地理；与 Vertex 的“系统图”是两种互补可视化范式
Machinery	流程图/Process Mining	概念相似（基于 Ontology 画图），但 Machinery 是 BPMN-like 静态流程，Vertex 是动态 what-if

一句话定位：**Vertex = Ontology-aware 系统图编辑器 + 时间序列时间线 + Scenario “什么如果” 沙箱 + Model Mesh（模型链）执行入口。**

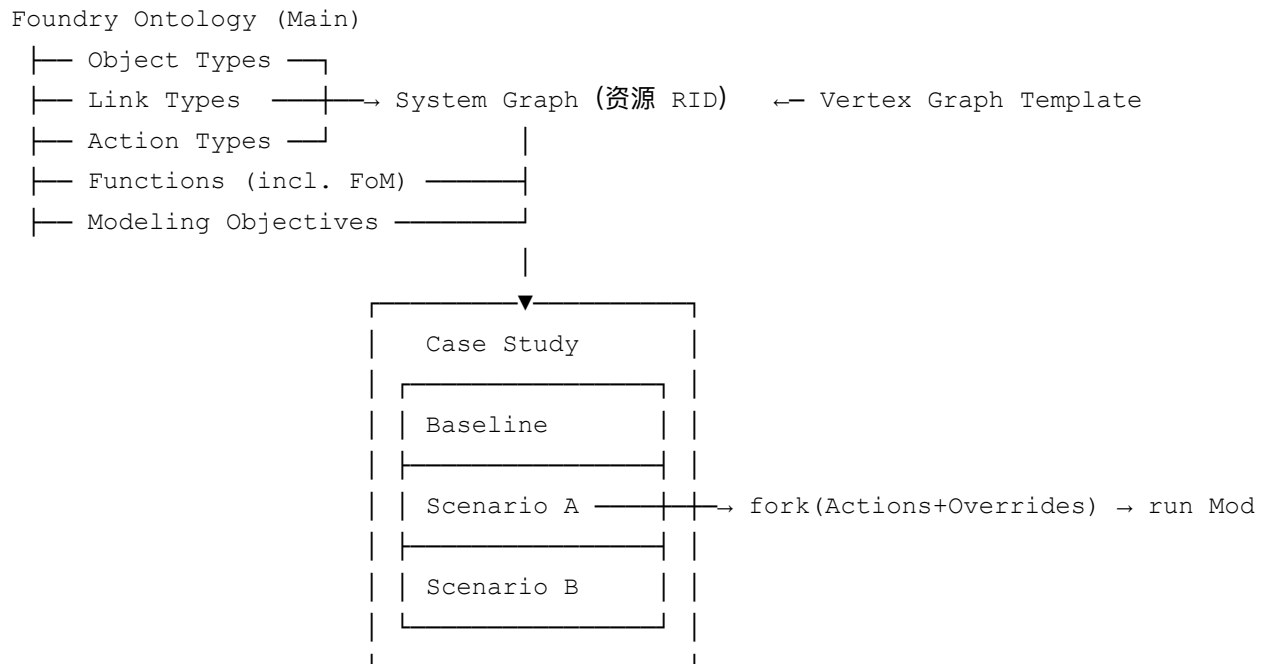
1.2 核心概念定义

- **System Graph（系统图）**：以 Ontology 对象为节点、以 Ontology 链接为边构建的可视化网络，是 Vertex 的主工作区。一个 System Graph 是一个独立的 Foundry 资源（具有 RID），可以保存、版本化、共享、嵌入 Workshop，并可以[“模板化（Graph Template）”](#)以便参数化生成。
- **Object Node（对象节点）**：图中每个节点对应 Ontology 中的一个对象实例（object instance），承载该对象的属性、关联事件、相关时间序列。节点的视觉样式可被 [“Layer Styling”](#) 按属性/时间序列值动态着色。
- **Model Node / Functions on Models**：在 Scenario 上下文中，模型本身**不是图上的节点**，而是 Scenario pane（侧栏表格）里的一行配置。它代表一个被绑定到 Ontology 的 [Model Function](#)（自动生成的 wrapper，封装了一个 Live Model Deployment），其 input/output parameters 被映射到对象属性/时间序列。
- **Scenario（场景）**：一组“Actions + Overrides + 选定模型”的集合，是 Ontology 在沙箱中

的一个分叉 (fork/branch) 。运行 Scenario 会基于该分叉的数据状态计算模型输出 ([Workshop Scenarios concepts](#)) 。Scenario 一旦创建即 **immutable**——要修改必须新建或 duplicate。

- **Case Study (案例研究)** : Scenario 的容器/分组。一个 Case Study 包含一个 Baseline + 多个并列 Scenario, 便于横向比较 ([Vertex scenarios getting started](#)) 。Case Study 可重命名、保存。
- **Override (覆盖值)** : 用户在 Scenario 表格中对某个 Model Input (标量、对象属性或时间序列点) 手工填入的“假设值”, 覆盖来自 Ontology 的当前实际值。被覆盖的格子在 UI 上高亮 ([Scenario options](#)) 。
- **Baseline (基线)** : 在含有 Actions 或 Overrides 的 Scenario 旁, 平台可被配置为自动跑一次“无修改”的运行作为对照基线。这是 Scenario options 中的一个开关。
- **Action (动作)** : Ontology Action Type 的实例化执行。在 Scenario 中, Action 不写回真实 Ontology, 而是写回 Scenario 内部的“fork”。一个 Action 可以是 **function-backed** (其后端是 Functions on Models, 从而把模型推理转化为对对象属性的写入) 。

1.3 关系图谱 (概念关系)



1.4 “Digital Twin” 在 Vertex 中的含义

官方明确表述: “Vertex allows you to **visualize and quantify cause and effect across the digital twin** of your real-world organization.” 在 Palantir 的语境里, “Digital Twin”并非工业仿真中那种基于物理方程的高保真模型, 而是:

1. **本体即孪生**：Ontology 本身就是企业的数字孪生——对象 = 实体（设备、订单、客户、井口、航班）、链接 = 关系（属于、依赖、流向）、Action = 决策、Function = 业务规则（[Ontology overview](#)、[Why create an Ontology](#)）。
2. **实时图视**：System Graph 是这个孪生在某一视角下的“切片”。通过 **Time Selection bar**（[顶部时间选择条](#)）+ **time series polling interval**（Control Panel 配置的轮询秒数，决定 live mode 多久拉一次时序），实现接近实时的状态反映。
3. **simulation mesh / model mesh**（来自 [Vertex for Oil & Gas 白皮书](#)）：多个绑定到对象类型的模型被自动链接（一个模型的输出经由 Ontology 属性映射成为下一个模型的输入），形成一个能跨越价值链的端到端可执行网络。

1.5 “What if” 核心用例

官方与白皮书列举（[Vertex example use cases](#)）：

- **油气**：井口设定点优化——评估数千种 setpoint，识别全局最优 BOE/碳排放折中；管线、Topside、运输网络对该设定点的影响通过 model chain 一并算出。
- **供应链**：原料中断时基于 BOM 数字孪生评估生产计划重排，订单履约风险。
- **航空运营**（官方示例数据集）：模拟某机场延误事件如何沿“Flight → Route → Aircraft → Airport”图传播。
- **生产排程**：工单设定变更前预演产能瓶颈。
- **碳/合规情景**：在多个 ESG 假设下重算 KPI。

二、System Graph 架构

2.1 数据模型：节点与边的定义

System Graph 的核心数据模型由四类配置项组成（基于 [Vertex 官方 docs](#) 推断的内部数据模型）：

```
// 简化的 SystemGraph 资源 JSON Schema（基于官方行为推断）【部分推测】
{
  "rid": "ri.vertex.main.graph.<uuid>",
  "name": "JFK Hub Operations",
  "versioned": true,
  "version": 7,
  "ontologyRid": "ri.ontology.main.ontology.<uuid>",
  "layers": [
```

```

{
  "id": "layer-airport",
  "objectTypeRid": "ri.ontology....object-type.Airport",
  "filter": { /* ObjectSet definition */ },
  "styling": {
    "fillColor": { "by": "property", "property": "alertScore", "scale": "rain" },
    "subtitle": { "property": "name" },
    "extendedLabels": [
      { "kind": "property", "property": "onTimePct" },
      { "kind": "timeSeries", "tsProperty": "passengerThroughput",
        "aggregation": "AVG", "windowMode": "selectedTimeWindow" },
      { "kind": "measure", "measure": "alertCount" }
    ],
    "badges": [{ "kind": "linkedEvents", "objectType": "FlightDelay" }],
    "displayAs": "card" // 或 "node"
  },
  "positions": {
    "<objectRid-1>": { "x": 240.5, "y": -118.0, "pinned": true },
    "<objectRid-2>": { "x": 412.0, "y": 12.5 }
  }
}
],

"edges": [
  {
    "linkTypeRid": "ri.ontology....link-type.airport-route-airport",
    "direction": "primary", // primary | undirectional | bidirectional
    "styling": { "colorBy": "objectType", "merge": true /* link merging */ }
  }
],

"savedSelections": [
  { "id": "sel-1", "name": "Hubs", "color": "#3FB36F",
    "objectRids": ["...", "..."], "showBorder": true }
],

"timeSettings": {
  "defaultWindowDays": 30,
  "selectedTime": "2026-05-14T00:00:00Z",
  "compareWindows": []
},

"scenarios": {
  "caseStudies": [ /* see §4 */ ]
}
}

```

构图方式（官方明示的三种来源）：

1. **手工**：从 + Add objects 搜索对话框检索对象类型/对象实例加入；右键 → [Search Around](#) 沿链接扩张子图；可通过“Search Around 函数”自定义“两跳/事件中继边”的探索逻辑。
2. **URL 参数**（ /workspace/vertex/graph/create ）： objectRid 、 objectSetRid 、 searchAroundFnRid 、 selectObjectRid 、 startTime/endTime/selectedTime （[generate graph from apps](#)）。
3. **Graph Template**： [把已有图保存为模板](#)，参数化（object 参数 + 非对象参数 + 关联 Search Around）。在 Workshop widget 中传入参数生成图。

Vertex 还要求每个 Link Type 在 Ontology 中带上 Vertex-specific type-classes 之一：

vertex:link_primary_direction 、 vertex:link_undirectional 、 vertex:link_bidirectional ，决定边箭头方向（[Object and edge display options](#)）。

2.2 节点 Extended Labels

每个节点除了图标 + 标题 + 副标题（subtitle，单属性）外，可挂载多个“扩展标签（Extended Labels / Readouts）”。三种来源：

- **Object Property**：直接属性值；
- **Time Series Reading**：基于当前 Time Selection 的聚合值（窗口内 AVG / LAST / MAX 等）；smoothing 可配置为按“X 分钟”做平滑（[Explore related time series](#)）；
- **Measure / Derived Property Function**：通过 Foundry Function 计算得出的衍生值（典型例子：右键 → 给机场节点加一个“totalAlerts = $\sum$ relatedRoute.alertCount”的派生函数）。

关键交互：Scenario 运行后，Extended Label 自动显示“baseline vs simulated”对比读数。

2.3 布局算法

官方 toolbar 提供的预定义布局（[Object and edge display options](#)）：

- **Auto / Circular**：无高级参数。
- **Hierarchical**：可配置 reverse 翻转方向、指定 Root nodes；自动按层次排列。
- 其他（force-directed / grid 类，名称未在 docs 显式枚举）：通过 gear 图标进入高级面板。

手动调整：用户可拖拽节点；位置作为 positions 中的 {x, y, pinned} 持久化到 Graph 资源。

2.4 持久化

- 每个 Graph 是 Foundry 顶级资源（RID 形如 ri.vertex.main.graph.<uuid>），存放于

Project 文件夹下，受标准权限控制（[Save, share, and collaborate](#)）。

- 支持 **versioned graph**：开启后每次保存创建新版本，可在 Graph History 侧栏查看历史，旧版本只读模式可访问。
- 支持 **Duplicate**（克隆）和 **Save as Template**（模板化）。

2.5 图交互

- 缩放/平移：鼠标滚轮 + 拖拽空白处。
 - 节点选择：单击 / 框选；选中的对象自动打开右侧 selection sidebar 显示属性、Series、Linked Events。
 - 展开/折叠：通过 right-click → [Search Around](#) 添加邻居；通过 **link merging**（Vertex 设置）把多条同向同类型边合并成一条粗边并显示数量。
 - 筛选：左侧 Histogram 面板（按属性直方图过滤）；顶部 Time filter（按 Shift+drag 选时间区间，时间外的节点变淡）；Saved selections（彩色边框分组）。
 - **Live mode**：根据 ["Time series polling interval"](#)（Control Panel，默认秒级）周期性刷新时间序列读数。
-

三、模型集成与配置

3.1 模型来源

Vertex 接入模型有两条路径，按官方“sunset”提示，第二条是推荐方向：

Path A (Legacy / Sunset)：通过 **Modeling Objectives** 把模型绑定到 Ontology

- 要求：模型已提交到 Objective 且配置了 objective API，并有一个 healthy Live deployment（PRODUCTION 标签）（[Workshop scenarios select a model](#)）。
- Vertex 中 `Add new model` 直接搜索绑定的 Model，选择 model version + configuration version。

Path B (推荐)：**Functions on Models → Function-backed Action**

- 把 Live Deployment 自动包装成 [Model Function](#)（同输入输出 API，TypeScript v1/v2 或 Python 均支持），把它 import 到 Functions repo，写一个 `@function` 包装该 Model Function 并加入自定义业务逻辑（如把对象属性映射成模型输入），最后注册为 **Function-backed Action**（[Configure chained models \(Sunset\)](#)）。
- 在 Vertex 中只用 Action（不直接选 Model）。

Path B 的代码片段（官方 Python 示例）：

```
@function(beta=True)
def predict_flight_delays(flight: Flight) -> Double:
    model_output = FoundryClient().ontology.queries.flight_model_deployment(
        df_in=[{"lastArrivalTime": flight.last_arrival_time,
                "lastExpectedArrivalTime": flight.last_expected_arrival_time}],
    )
    return model_output.df_out[0].prediction
```

3.2 模型参数到 Ontology 属性/时间序列的映射

在 Scenario pane 中通过 **+ Add input or output** 进行：

- 候选项包括：individual time series（对象的时序属性）、object property、measure。
- **Add all parameters**：把模型预配置的所有 I/O 一次性加入（这一映射在 Modeling Objective 端做了对象类型对齐，或在 Function 参数签名中表达）。
- Function-backed Action 路径下，参数映射由 Action 的 parameter 配置（每个 parameter 关联到一个对象属性或时间序列点）。

Model Function 的类型限制（[官方明示](#)）：仅允许 primitive scalars / Optional / Collection，不允许 Aggregation、Ontology object 类型、Notification（输入参数必须按属性逐一拆开）。输出必须是 anonymous custom type，字段都是允许的原始类型。

3.3 模型链式编排（Chaining）

机制（[官方在 chained-models 文档明示](#)，亦标记为 sunset，但 Function-Action 模式下“chain”由 Ontology 属性写回自然形成）：

1. 在同一个 Scenario 中加入多个模型；
2. 任何一个模型的某个 **output parameter** 写到对象属性 A；
3. 下游模型的某个 **input parameter** 读取属性 A；
4. Vertex 自动按依赖图前向传播：上游 output → Ontology 属性（在 Scenario fork 中）→ 下游 input。

前端可视化方式：Scenario pane 是一个**纵向表格**，每行一个 model；列显示其 inputs/outputs。被链路传递影响的输入会被高亮，表明它的当前值来自上游模型而不是 Ontology 实测。Vertex 文档原文：“Adding overrides in any of the selected model inputs will show impacted inputs in the chained models as you run ‘what if’ simulations.”

【推测】底层执行：在 Scenario fork 上拓扑排序，按 DAG 顺序串行/并行执行 Function 调用，

结果累加到 fork 的 edits 集合。

3.4 模型版本管理

- 每个 Model 在 Modeling Objective 中带版本号 (model version) + 配置版本号 (configuration version)。Vertex 的模型选择 UI 上有两个独立下拉框。
- Function-backed 路径下：通过 Function 的 semver tag (如 1.0.0) + Function version range dependency 来锁定。Vertex 默认拉 **latest published version** (Functions 通用规则)。

3.5 支持的模型类型

- **Foundry 内部模型**：原生 Modeling 模块 + Live Deployment。
- **外部模型 (API 调用)**：通过 Function 内 `make API calls from functions` 能力调用外部 REST 端点，再包装成 Action。也支持容器化模型 (Container Models) 部署后被 Function 调用。
- **LLM**：通过 Palantir-provided language models (GPT-4.1/GPT-4o 等) 在 Function 中调用 ([Language models in TypeScript v1 functions](#))；这种通常服务于 AIP Logic，而不是定量 Scenario 推演的主线。

3.6 Functions on Models → Action 的封装链条 (关键复刻要点)

```
Live Model Deployment
| 自动生成 wrapper
▼
Model Function (in Functions repo, typed I/O)
| import + 业务包装
▼
@function 自定义函数 (接收 Ontology Object 输入、做属性映射、调用 Model Function、写回属性)
| 注册
▼
Function-backed Action Type (在 Ontology Manager 配置 parameter 表单)
|
▼
出现在 Vertex Scenario 的 "Add Action" 下拉中
```

这一链条意味着：**Vertex 本身不直接调度 ML 推理**，它只调用 **Ontology Action**。Action 执行框架 (OSv2 Actions service) 负责评估 Function、维护 Ontology edits 的事务性。

四、Scenario 场景推演引擎

4.1 完整生命周期

1. 在 Case Study 中 + Add Scenario
2. 在该 Scenario 上展开 → Add Action (选择一个已发布的 Function-backed Action 或普通 Action)
 - 2.1 填写 Action parameters (关联对象 + 字段值) → Submit
 - 2.2 可重复添加多个 Action (单 Scenario 上限 50 个 Action)
3. + Add input or output (按需勾选要在表中显示的输入/输出参数; 可批量 Add all)
4. (可选) 在 input 单元格输入 override, 单元格高亮
5. Run
 - 5.1 平台为该 Scenario 创建一个 Ontology fork (含 edits)
 - 5.2 在 fork 上评估所有 Action 的 Function-backed 计算
 - 5.3 按 model 链 DAG 顺序产生 outputs, 写回 fork
 - 5.4 同时 (若开启 Run baseline) 跑一次无 override/Action 的副本作为对照
6. 完成 → 绿色 ✓ + 显示执行耗时
7. 在 System Graph 的 Extended Labels 上看到"baseline vs simulated"对比
8. 重复 1-7 创建多个 Scenario 横向比较
9. 重命名 Case Study / 保存 / 共享

参考: [Vertex Scenarios getting started](#)、[Scenario options](#)。

4.2 Action 的角色

- **预配置:** Action Type 必须先在 Ontology Manager 中定义 (包括 rules、submission criteria、parameter forms) 。
- **在 Scenario 中执行:** 与生产 Action 走的是同一 **Actions 服务**, 但其执行 context 标记为"作用于 Scenario fork", 因此:
 - 不会真正写回 Main Ontology;
 - 不触发 webhook、不发邮件、不调用外部 API;
 - 不会触发普通的 Action audit/notification side-effects (这一行为来自 [Foundry Branching](#) 对 Action on Branch 的限制, Workshop Scenario 同样遵循此约束) 。
- **Function-backed Action** 在此 context 等效于"模型预测 + 属性写回"——把 ML 输出落到 fork 上的对象属性。
- **限制:** 单 Scenario ≤ 50 Actions; 单 Scenario ≤ 30,000 edits; 对象集 load ≤ 10,000 (来自 [Workshop Scenario 文档](#); Vertex Scenario 沿用同一底层) 。

4.3 Override 机制

- **粒度:**
 - 标量 input: 直接覆盖一个单元格值;

- **对象属性**：覆盖某对象的某个属性（仅在 fork 内生效）；
- **时间序列**：可覆盖当前 **Selected Time** 所在窗口聚合后的值作为模型输入；不需要逐点修改整条序列。[Scenario options](#) 提到“smoothing over set periods (minutes)”决定时序如何被聚合后送入模型。
- **存储**：Override 与 Scenario 一同持久化在 Case Study 配置内（Scenario 资源是 immutable 的，所以 Override 一旦保存就和该 Scenario 绑死）。
- **UI 提示**：覆盖单元格在 Scenario table 中高亮；运行后下游受影响的 input 也会高亮，让用户看到传播路径。

4.4 运行引擎

【部分推测，基于公开行为】

- **同步还是异步**：UI 显示“执行耗时”且会显示绿色 ✓ → 异步任务，前端通过 polling 或 server-sent updates 等待。对单个 Scenario 的 Run，Vertex 把 fork 描述 + 选定 Models 提交给一个 Scenario 执行服务（与 Workshop Scenario 共享，详见第八章），由其调度 Functions / Actions 求值。
- **调度顺序**：按模型 input/output 与 Ontology 属性的依赖图做拓扑排序；同层可并发。
- **结果回写**：写到 Scenario fork（一个内存中或临时存储中的 Ontology edits 集合），UI 通过 Ontology read-with-scenario 模式拉取，使 Extended Labels 反映 fork 中的值。
- **隔离**：Scenario fork 不写真实 **Ontology**，除非用户在 Workshop 中显式调用 [Apply Scenario](#)（Workshop 特有功能；Vertex 端原生提供的是 **save Scenario / Save Case Study**，“apply”一般通过 Workshop 嵌入完成）。

4.5 多 Scenario 对比

- Scenario table 同时展示 Baseline + 各 Scenario 各列；
- 在 System Graph 的 Extended Labels 上可直接看 simulated vs baseline；
- [Workshop Scenario 文档](#) 明确：Object Table / XY Chart / Metric Card 都支持任意多个 Scenario 作为 layer 并列展示（Vertex 共享该能力但主要在 Scenario pane 内对比）。

4.6 沙箱隔离机制（关键复刻要点）

官方原文（[Workshop Scenario concepts](#)，与 Vertex Scenario 共享底层）：

“A Scenario is essentially a **‘fork’ or ‘branch’ of the data in your Ontology** generated by applying one or more actions. The fork contains only the edits or changes from the base Ontology including modified Object properties, created Objects, deleted Objects, created link types, and deleted link types. A Scenario is immutable once created.”

实现要点：

- 是轻量级 **delta**（增量）层而非整库 copy：仅存 edits。
 - **immutable**：保证可重复执行结果一致 → 类似 functional persistent data structure 或 Git tree。
 - 与 [Foundry Branching/Ontology Branches](#) 是两种不同机制：
 - Ontology Branches/Foundry Branches 是**长时跨多应用的** schema-level + data-level 分支（开发-测试-上线流程），存在 retention policy（不活跃自动关闭）。
 - Scenario 是**短时、单用户、纯数据 delta**的“假设态”，不修改 schema。
 - 【推测】内部由 Ontology service (OSv2) 提供 “read with overlay” 接口：每次对象查询附带一个 `scenarioRid`，服务在 base + delta 上合并视图返回。这与 Foundry 多项专利文档（“Simulated change of immutable objects during execution runtime”，[US 10,990,425](#) / [US 11,537,418](#)）的思路高度一致——immutable 节点 + simulated change apparatus + cached output。
-

五、时间序列处理

5.1 为何核心

Scenario 的最大价值是“预测未来”，而模型大多数是时序模型（demand forecast、process control、KPI 预测）。因此：

- Ontology 中**时序属性（time series property）** 是头等公民；
- 系统图整体由顶部 [Time Selection bar](#) 控制；可以拖动一根游标在过去/现在/未来移动，Extended Labels 与节点着色随之更新。

5.2 数据格式

【主要为推测，与 Foundry Time Series 服务的公开行为一致】

- 时间戳：毫秒精度 UNIX timestamp 或 ISO-8601；URL 同时接受这两种。
- 值类型：数值（Double 主流）、Boolean、可选字符串；可附带 quality flag。
- 元数据：单位、阈值（Vertex 在 Ontology Manager 的 “Vertex Capabilities” 部分[配置 Threshold](#) 用于事件着色）、source、采样率。

5.3 在 System Graph 上的可视化

- **Sparkline-like Extended Label**: 节点旁直接显示一个时序数值（窗口聚合后的标量）。
- **Series Panel（下方面板）**：选中节点 → Series tab → 把感兴趣的 series [“Open in series view”](#) → 底部出现完整时间线，可拖游标移动到任一时间点。
- **Timeline（事件 + 时序的统一视图）**：把事件画成时间条 bar，时序画成 line；支持 play 按钮自动滚动 + 速度控制 + 循环。
- **“Open in Quiver / Object Explorer”**：跳转到专业时序分析视图。

5.4 时间序列的 Override 交互

- 在 Scenario table 中：选中一个时序 input 单元格 → 输入新值（标量，代表选定时间窗口的聚合值，受 smoothing 配置影响）。
- **【推测】**通过“compare time windows”（顶部 + Compare button 可创建多个时间窗口对比读数）实现“假设未来某段时间窗口下的值”的对照。
- 文档未显示支持“在图表上拖拽曲线点修改”的精细化交互，而是以**窗口聚合后的标量覆盖**为主——这在工程实现上更简单。

5.5 历史 vs 预测 视觉区分

- 通过两个 [Time Selection 比较窗口](#)实现：过去窗口 vs 未来窗口。
- Threshold 配置可让超阈值时段着色（同一 series 不同段不同色）。
- [Events](#)（带 start/end 时间的对象类型）作为时间线上的色条覆盖，对未来的预测事件与历史事件可以用不同 Event 类型区分。
- “Time series missing data warning”：当某序列在选定时间附近没有近期观测时显示警告（[控制面板可调上限小时数](#)）。

六、前端 UI 与交互设计

6.1 整体页面布局

基于截图与文档描述，典型 Vertex 工作区分为：

TopBar: Save/Share Layout selector Time Selection bar Run			
Left side	CENTER: Graph Canvas (nodes + edges + extended labels)		Right: Selection sidebar

-bar	+ saved selections borders)	- Properties
		- Series
Lay-		- Linked Events
ers		- Derived Funcs
Sel		
Sear		
Hist		
Info		
BOTTOM: Series Panel / Timeline (可折叠)		
Scenario Pane (可折叠到侧栏; 表格式: rows = Actions/Models, cols = inputs/outputs + override; per-scenario column)		

6.2 前端组件库

【推测】Palantir 整体平台使用其自研开源组件库 [Blueprint.js](#)（React 组件、Canvas/SVG icons、Popover、Drawer、Tag、HTMLTable 等）。Vertex 中常见 UI 元素（侧栏可折叠、time scrubber、resource picker、context menu）与 Blueprint 风格一致。

6.3 System Graph 渲染技术

【推测，文档未明示】综合截图特征 + Palantir 量级（数千节点）+ link merging + 自定义节点卡片的需求，Vertex 的图渲染最可能采用 **Canvas2D 自研渲染层 + 自研力学/层次布局**：

- WebGL (Sigma.js 风格) 适合极大规模（10k+），但**自定义节点 = 富 HTML 卡片**（Object card 显示属性、Sparkline、徽标）在 WebGL 中实现复杂；
- 因此推测的混合方案：Canvas 绘制边 + 节点底版，DOM/HTML 绝对定位渲染节点卡片（与 Blueprint 集成），SVG 用于 overlay（saved selection 彩色边框）。
- 布局算法：Hierarchical/Circular/Force 等是经典算法，可能基于 dagre/elkjs/d3-force 思想自研。

6.4 Scenario 面板的交互模式

- 类似 **Excel** 的网格：行 = Actions/Models，列 = inputs（覆盖列）/ outputs（计算列）/ baseline-comparison 列。
- 顶部按钮： + Add Action 、 + Add Scenario 、 + Add input or output 、 Run 。
- 每个 Scenario 折叠头部显示运行状态（✓ + 耗时 / spinner / ✕ + error）。
- **单元格交互**：input 单元格 click → 输入框 → blur 后高亮表明 override；output 只读，可悬停看 baseline 对比。

6.5 拖拽与跨应用集成

- **从 Object Explorer → Vertex**: 通过 URL 参数 `objectSetRid` 传入对象集; 目前未见显式“拖拽”机制, 主要是 deep-link ([generate graph from apps](#))。
- **Workshop 嵌入**: [Vertex Graph Widget](#) 在 Workshop 中作为 widget, 支持参数化 + Override graph RID + 触发 Save action 把当前 graph state 存为资源。

6.6 协作

- **共享链接**: Get quick share link (同组织只读); 跨组织需要共享 Project ([Save, share, and collaborate](#))。
 - **多人同时编辑**: 文档未明确“实时协同”。**versioned graphs** 通过版本历史防冲突——【推测】采用 last-write-wins 或乐观锁, 不是 OT/CRDT 实时协作。
 - 缺访问权时仍能看到 graph 结构但看不到具体数据值 (隐私保护)。
-

七、后端架构与数据流 (部分推测)

7.1 服务架构

【推测】Vertex 后端由若干微服务组成, 部署于 Foundry 平台内:

- **Vertex API/Graph service**: CRUD System Graph 资源 (layout、styling、saved selections)。
- **Vertex Scenario service** (很可能与 Workshop Scenario service 共享): 管理 Case Study/Scenario fork、调度 Function/Action 执行、聚合输出。
- **Functions runtime** (Foundry 通用): 执行 TypeScript/Python Functions、调用 Model Function。
- **OSv2 (Object Storage v2)**: 提供 Ontology read/write 接口, 支持 “with scenario” overlay 读模式。
- **Time Series service** (Foundry 通用): 提供时序点查/聚合。
- **Resource service / Compass**: 管理 RID、权限、Project 归属、版本历史。

7.2 Scenario 数据存储

【推测】Scenario edits 以 “edit log” 形式存储, 结构类似:

```
{
```

```

"scenarioRid": "ri.scenario.main.scenario.<uuid>",
"parentOntologyState": "ri.ontology....ontology@<commit>",
"edits": [
  { "kind": "modifyProperty",
    "objectRid": "ri....airport.JFK",
    "property": "capacity", "value": 1200 },
  { "kind": "createObject", "objectType": "...Order", "properties": {...} },
  { "kind": "deleteObject", "objectRid": "..." },
  { "kind": "createLink", "linkTypeRid": "...", "src": "...", "dst": "..." }
],
"actionExecutions": [
  { "actionTypeRid": "...", "parameters": {...}, "appliedAt": "..." }
],
"overrides": [
  { "modelRid": "...", "parameter": "demand_input",
    "objectRid": "...", "value": 950, "appliedTimestamp": "2026-05-15T..." }
],
"results": [
  { "modelRid": "...", "outputParameter": "yield", "value": 0.87,
    "objectRid": "..." }
],
"immutable": true,
"createdBy": "...", "createdAt": "..."
}

```

存储介质很可能是 Foundry 内部的 Atlas/KV 存储（非用户可见数据集）。

7.3 模型执行层

Vertex 不维护自己的训练/推理基础设施；它委托：

- **Modeling 模块** 的 Live Deployment（容器化运行的模型服务）；
- **Functions runtime**（轻量 TS/Python 执行环境）做 wrapper 和编排。

7.4 System Graph 配置 CRUD API（推测端点设计，可复刻参考）

GET	/api/vertex/v1/graphs/{rid}	# 读图
POST	/api/vertex/v1/graphs	# 新建
PUT	/api/vertex/v1/graphs/{rid}	# 全量保存（创建新版本，若 versi
PATCH	/api/vertex/v1/graphs/{rid}/layout	# 增量保存位置
POST	/api/vertex/v1/graphs/{rid}/duplicate	# 克隆
POST	/api/vertex/v1/graphs/{rid}/save-as-template	
GET	/api/vertex/v1/graphs/{rid}/history	# 版本历史
GET	/api/vertex/v1/graphs/{rid}/versions/{n}	# 指定版本（只读）


```

POST    /api/vertex/v1/graphs/{rid}/case-studies
POST    /api/vertex/v1/case-studies/{rid}/scenarios
POST    /api/vertex/v1/scenarios/{rid}/actions
POST    /api/vertex/v1/scenarios/{rid}/overrides
POST    /api/vertex/v1/scenarios/{rid}/run          # 异步：返回 jobId
GET     /api/vertex/v1/scenarios/{rid}/run/{jobId}  # 拉取运行状态/结果

GET     /api/vertex/v1/search-arounds/{fnRid}/execute # 调用 Search Around 函数

```

7.5 与 Ontology 服务的交互

- 读对象/属性/链接：使用内部 Ontology API（在公开侧对应 **OSv2 / Ontology SDK / OSDK**）。所有读操作都可以带 `scenarioRid` 上下文做 fork-aware 查询。
- 时间序列：通过 Ontology 上的 time series property → Time Series service。
- 提交 Action：通过 Actions service 调用，传入 `branch / scenario context` 标识。

7.6 权限模型

- **Graph 资源**：受 Project-level + Resource-level Roles 控制（与所有 Foundry 资源一致），用 Compass / Approvals 框架。
- **共享时**：分享链接不会自动授予访问对象/时序的权限；缺权限的用户看到结构但不见值。
- **Action 执行**：受 Action Type 的 permission policy 控制，并在 Scenario context 下仍校验“用户是否能对此对象执行此 Action”（虽然不写回 Main，但权限语义保留）。
- **Vertex Admin**：[Control Panel 配置](#)（默认时间窗、轮询间隔、search around limits、active icon categories 等）。

7.7 实时数据更新

- **Live mode** = 定时轮询，间隔由 `Time series polling interval`（Control Panel，单位秒）配置。
- 未见 WebSocket 推送的明确说法【推测】：Vertex 选择 polling 而非 WebSocket 大概率因为 (a) 数据量按节点 × 时序需要分页拉取；(b) Foundry 普遍依赖 Compass 的 polling/event-stream 机制做资源更新提示。
- “Time series missing data warning” 阈值（hours）：当观测延迟超过阈值显示警告。

八、与 Ontology Scenarios（非 Vertex）的关系

8.1 三种 Scenario / Branching 概念辨析

概念	持续时间	影响范围	是否修改 schema	主要用途
Foundry Branching./Ontology Branches	较长 (天-周)	Ontology schema + 数据集 + Functions(v1) + Workshop 模块 + Actions	可修改 schema	workflow 变更的开发-评审-部署
Workshop Scenario	短 (用户会话内)	仅 Ontology 数据 delta (property/object/link edits)	不可改 schema	应用内 what-if, 可显式 Apply 写回 Main
Vertex Scenario	短 (用户会话内)	同 Workshop Scenario	同上	系统图上的 what-if、模型链推演

8.2 Vertex Scenario 与 Workshop Scenario 是同一套底层

官方原文都使用相同定义"a 'fork' or 'branch' of the data in your Ontology", 限额 (≤50 Actions、≤30k edits、≤10k objects load) 也一致, 且都依赖 Functions on Models + Function-backed Actions 来引入模型推理。**复刻时只需要实现一套 Scenario 服务**, 再分别接入两种 UI 上层。Workshop 端比 Vertex 多一个 [Apply Scenario](#) 按钮 (把 fork 内的 Action 集合事务性提交到 Main, 并可级联触发 follow-up Action) 。

8.3 AIP Logic 与 Vertex

- AIP Logic 是基于 LLM 的"块编排"开发环境 (无代码声明 prompt + LLM block + Ontology Read/Edit block + Function 调用) 。
- [AIP Evals](#) (评估子模块) 会在 Ontology Edits 测试中使用一个"Ontology Simulation", **复用 Scenario 的沙箱机制**: 每个 test case 在隔离的 Ontology fork 中跑一次 Logic function, 验证 edits 是否符合预期。官方原文: "each test case is executed in an Ontology simulation. This ensures that the actual Ontology remains unchanged during testing and evaluation."
- 因此 AIP Logic 与 Vertex 在底层共用 Scenario 沙箱设施, 但用法不同: Vertex 用于人类驱动连续 what-if 调优, AIP Evals 用于自动化测试 LLM-driven 函数。

九、竞品对比

9.1 与传统数字孪生平台

平台	模型基础	模拟方式	优劣对比 Vertex
Azure Digital Twins	DTDL (Digital Twin Definition Language) + Twin Graph	主要用于 IoT 状态镜像, simulation 需用户自接 Azure Functions/Stream Analytics	DTDL 更标准但缺乏开箱即用的“模型链 + Scenario fork”; Vertex 把模型+Action+Ontology 紧耦合可直接做端到端 what-if
AWS IoT TwinMaker	Knowledge Graph + Scene Composer (3D 视图)	集成 SiteWise/IoT Core 读时序、Grafana 出图; 模拟靠 Lambda	强在 3D 场景与 AWS 生态集成; 弱在缺少 Vertex 那种“Scenario+Override+模型链”的产品化抽象
Siemens MindSphere / Industrial Edge	工业资产模型 + 物理仿真	偏 OT/物理仿真 (CFD、FEM)	物理保真度高于 Vertex, 但难做跨业务领域 (财务、订单、客户) 的统一孪生

Vertex 的独特点在于: **Ontology** 把业务对象 (订单/客户/供应商) 和工程对象 (设备/管道) 放在同一个孪生里, 且 Scenario fork 是 Ontology-level 而不是模型 level, 因此一个 Scenario 内可以同时变动业务规则和工程参数。

9.2 与 Scenario Planning 工具

工具	主体范式	区别
Anaplan	多维 Hyperblock 公式模型	强在 FP&A 用例 (金融规划、HR); 模型即 spreadsheet, 无 ML 推理与图谱
Board	OLAP cube + 表单	同上, 偏 BI/Planning
IBM Planning Analytics (TM1)	TM1 cube 内存数据库	强在快速重算大型立方体; 缺乏 Ontology 图、缺乏 ML 模型链

Vertex 的差异: **图 (Ontology graph) + 模型链 (model mesh) + Action (决策落地)** 三位

一体，传统 Planning 工具基本只有“维度计算”。

9.3 开源方案

目前没有一个开源项目把“Ontology + ML 模型 + Scenario fork + 时序 + 系统图 UI”完整集成。可分拆复用的组件：

- **Ontology 层**：Apache Jena/RDF4J（语义太重）；或基于关系数据库 + 元数据表自建（Vertex 风格更接近后者）。OSDK（Palantir 开源的 Ontology SDK 客户端）只是 client 库。
- **图可视化**：[Cytoscape.js](#)（功能丰富，~10k 节点可用）、[Sigma.js](#) + [Graphology](#)（WebGL，10k-100k 节点）、Linkurious Ogmia（商业，性能最强）、[G6](#)（蚂蚁开源，扩展性好）、yFiles/Keylines（商业）。
- **工作流引擎**：[Temporal](#)、[Prefect](#)、Apache Airflow、Dagster。
- **时序数据库**：[TimescaleDB](#)、[QuestDB](#)、InfluxDB、Apache IoTDB。
- **Scenario fork 机制**：可借鉴 [Dolt](#)（Git-style SQL DB）、Pachyderm（数据版本化）、Iceberg/Delta Lake 的 snapshot 模型，但都不天然支持“细粒度对象 edit overlay”。

9.4 Vertex 差异化总结

1. **Ontology-native**：Scenario fork 直接作用在企业的“决策孪生”上，而不是被迫先做数据集 ETL。
2. **Model Mesh**：模型间的链式编排由 Ontology 属性映射自然实现，无需 DAG 编辑器。
3. **Action 写回闭环**：what-if 后可直接通过 Workshop 的 `Apply Scenario` 转生产决策——竞品大多到“建议”为止，无标准化的回执路径。
4. **统一权限/审计**：Scenario 内的 Action 仍走平台权限链，避免影子决策。
5. **多视角融合**：时序、事件、地理（Map）、流程（Machinery）共享同一 Ontology。

十、复刻技术方案建议

10.1 System Graph 可视化层

推荐栈：React + [Sigma.js](#) + [Graphology](#) + @react-sigma/core 作为基础（WebGL，10k+ 节点流畅），叠加 **HTML/DOM overlay 实现“对象卡片式节点”**（在 Sigma 的 stage 之上用绝对定位 div 显示节点 Extended Labels/Sparkline，跟随相机变换）。

- 布局：

- Hierarchical → 用 [dagre](#) 或 [elkjs](#);
- Force-directed → graphology-layout-forceatlas2 (与 Sigma 配套);
- Circular → graphology-layout-circular;
- Auto → 启发式选择 (节点数 < 100 用 force, 否则 hierarchical) 。
- 边合并: Graphology 自定义 reducer, 把同类型同向多边压成一条加权边。
- Sparkline: 用 [uPlot](#) (轻量、Canvas、亚毫秒级), 嵌进节点 DOM overlay。
- 替代方案: 超大规模 (>100k) → [Cosmograph](#); 需要更丰富开箱 UX → 商业 Ogma。

10.2 Scenario 引擎: 沙箱隔离

强烈推荐: 应用层 delta overlay (即 Vertex 实际做法的开源复刻), 具体方案:

- **数据模型:** 用 PostgreSQL 存"对象"和"链接"主表 + 一张 scenario_edits 表:

```
CREATE TABLE scenario_edits (
  scenario_id    UUID,
  seq            BIGSERIAL,
  op             TEXT CHECK (op IN ('modify','create','delete','link_add','li
  object_id      TEXT,
  property       TEXT,
  new_value      JSONB,
  link_type      TEXT,
  src_id         TEXT,
  dst_id         TEXT,
  created_at     TIMESTAMPTZ DEFAULT now(),
  PRIMARY KEY (scenario_id, seq)
);
CREATE INDEX ON scenario_edits (scenario_id, object_id);
```

- **读路径:** Ontology read API 接收可选 scenario_id, 对结果应用该 scenario 的 edits (按 seq 顺序 fold)。可在内存缓存"已物化 fork 视图"。
- **不推荐 DB-level branching** (Dolt/Postgres logical replication slot): 实现成本高且对查询性能侵扰大。
- **不推荐 Git-like 真正 commit-on-fork:** edits 数据量小、生命周期短, 无须 Merkle DAG。

10.3 模型执行调度

推荐: [Temporal](#)

- 每个 Scenario Run 是一个 Workflow;

- 每个 model/Function 是一个 Activity；
- 依赖关系由 Workflow 代码显式拓扑排序（或借助 `temporal.workflow.start_activity` 的 promise 自然并发）；
- 重试、超时、cancellation、Worker 容错都开箱即用。
- 备选： [Prefect 2](#) / [Dagster](#)（Python 生态友好，定时调度强）；自建 Celery + Redis（轻但缺少 visibility）。

模型推理层：

- Python 模型 → FastAPI + [Triton Inference Server](#)（GPU）或纯 FastAPI（CPU） + `pydantic` 类型化输入输出；
- 通过 OpenAPI/gRPC 暴露，被“Function wrapper”调用。
- “Function wrapper”复刻：在 Temporal Activity 内接到 Foundry-style “Function” abstraction，类型签名与 Model Function 一致。

10.4 时间序列存储与查询

推荐： [TimescaleDB](#)（基于 PostgreSQL，hypertable + continuous aggregates + 与对象表 join 自然）。

- 单一 PG 数据库 = 对象主存 + scenario_edits + 时序 → 大幅降低运维复杂度。
- 备选：
 - [QuestDB](#)：极致写入性能，SQL 友好；外置同步成本。
 - [InfluxDB v3 \(IOx\)](#)：列存 + Arrow，分析型 OK，但 schema 灵活性弱。
 - [ClickHouse](#)：超大规模分析，但 schema/索引代价高。
- 查询场景核心是“对象 + 时间窗口 → 聚合标量”，TimescaleDB 的 `time_bucket + AVG/LAST` 一句话搞定。

10.5 与已有 OSDK 复刻项目的集成

假设你已有一套 OSDK-compatible 的 Ontology 服务（对象类型/属性/链接/Action 已实现）：

- **新增 Scenario 上下文头**：所有 Ontology 读 API 接受 `X-Scenario-Id` header，按 §10.2 应用 overlay。
- **Action service**：增加 `scope = scenario | main` 参数。scenario 模式下：跳过 webhook、邮件、外部 API 调用；edits 写入 `scenario_edits` 而非主表；触发 audit log（带 scenario 标识）。

- **Functions runtime**: 在 sandbox 里注入一个 client，其所有读写都被路由到带 scenario header 的 endpoint。可参考 OSDK 的 client builder 模式。
- **Vertex Graph service** 作为独立服务，调用 Ontology service 拉数据；Scenario service 维护 fork；二者通过 RID 关联。

10.6 前端技术栈

层	推荐
框架	React 18 + TypeScript
路由	React Router
状态管理	Zustand （轻量、graph state 巨大时易于切片）或 Redux Toolkit + RTK Query
服务调用	TanStack Query （缓存、polling、suspense）
UI 组件	Blueprint.js （同 Palantir 风格）或 Radix UI + 自研主题
图渲染	Sigma.js + Graphology + @react-sigma/core ，DOM overlay 渲染节点卡片
图表/ Sparkline	uPlot （高性能）+ Visx （D3-on-React，复杂时序）
拖拽	dnd-kit （侧栏拖到画布）
时间轴	自研基于 uPlot/d3-scale；事件条用 <code><rect></code> SVG overlay
长任务通信	SSE（Server-Sent Events）从后端推 Run 状态
表单/输入	React Hook Form + Zod schema 校验

10.7 MVP 功能清单与优先级

P0（必须，4-6 周）：

1. Ontology read API（对象、链接、属性、时间序列窗口聚合查询），含 X-Scenario-Id overlay 机制；
2. System Graph 资源 CRUD（创建/读/保存/版本化）；
3. 画布渲染：节点 + 边 + 单属性 subtitle + 单时序 sparkline；力学 + 层次布局；缩放/平移/选择；
4. Time Selection bar（单一窗口）+ time series polling；

5. Scenario 数据模型 + Override (仅标量 input) + Run 同步执行单个 Function;
6. Scenario table UI (行=Action/Function, 列=inputs/outputs, 覆盖单元格高亮);
7. Baseline 对比读数显示在 Extended Label。

P1 (次必要, 6-8 周) : 8. Action 集成 (Function-backed Action 调用, Scenario context 隔离); 9. Model 链: 多模型自动按 Ontology 属性依赖拓扑排序异步执行 (接入 Temporal); 10. 多 Scenario 并列 + Case Study 容器; 11. Search Around (基于链接的邻居展开, 含 Foundry Function 自定义 search around); 12. Layer styling (按属性/时序着色, extended labels 多读数); 13. Saved selections + Histogram 过滤。

P2 (增强, 4-6 周) : 14. Graph Template + URL 参数生成图; 15. Event 时间线 (事件 bar + threshold 着色) + Compare 时间窗口; 16. 异步运行 + SSE 状态推送 + 错误重试可视化; 17. 共享链接 + 权限模型完整集成; 18. 嵌入 widget (在你的 Workshop-like app 中)。

P3 (高阶, 6+ 周) : 19. Graph Template 参数化 search around; 20. Live mode 与实时告警 badge; 21. 协作 (最简单的乐观锁 + 版本冲突提示); 22. AIP-style LLM Scenario copilot (用 LLM 推荐 override 取值范围、解读结果)。

附录 A: 复刻团队第一周可立刻动手的具体任务

1. **数据模型 PoC:** 建一个 PG schema (objects、links、properties、time_series_hypertable、scenario、scenario_edits), 用 5-10 个对象类型 + 100 个对象 + 1 条时序填充示例数据。
2. **Read-with-overlay PoC:** 写一个 `GET /objects/{id}?scenarioId=...` 端点, 验证 fold edits 正确性, benchmark 单次读延迟 < 20ms。
3. **画布 PoC:** 用 react-sigma 渲染 200 个节点 + 500 条边, 绑定时序属性着色, 加一个 time slider 改变颜色。
4. **Function runtime PoC:** 用 Python + FastAPI 跑一个简单的 sklearn 模型, 包装成“Function”接口 (输入 = 对象 + 属性, 输出 = 数值), 在 Temporal Activity 内调用。
5. **Scenario Run PoC:** 构造一个 case study 含 1 个 Override + 1 个 Function-backed Action, 端到端跑通“覆盖→Function 执行→edit 写 scenario_edits→read overlay 反映新值”。

完成这五项即可开始迭代 P0 完整版。

附录 B: 关键 API 端点设计示例 (节选)


```
# Scenario lifecycle
POST    /scenarios
  body: { caseStudyId, name, parentOntologyCommit }
  → { scenarioId, status: "draft" }

POST    /scenarios/{id}/actions
  body: { actionTypeRid, parameters: { ... } }
  → { actionExecutionId }

POST    /scenarios/{id}/overrides
  body: { modelRid, parameter, objectRid, value, atTimestamp? }

POST    /scenarios/{id}/run
  body: { runBaseline: true }
  → { jobId } (202 Accepted)

GET      /scenarios/{id}/run/{jobId}/stream      # SSE
  events: { type: "progress", model, pct }
           { type: "result",   model, output }
           { type: "done",    durationMs }
           { type: "error",   model, message }

GET      /scenarios/{id}/diff
  → { editedObjects: [...], createdObjects, deletedObjects, deltas: [...] }

# Ontology read with scenario overlay (per-request)
GET      /objects/{rid}?scenarioId=...&time=...
GET      /objects/{rid}/timeseries/{prop}?from=...&to=...&agg=AVG&scenarioId=...
```

附录 C：风险与未解项

- **拖拽修改时序曲线**：用户问题中提到了，但官方文档**未明确**支持在图表上拖拽点修改预测值；Vertex 当前以“窗口聚合标量 override”为主。复刻时可考虑作为差异化增强，但需要谨慎设计与 Smoothing 配置的语义关系。
- **实时协作**：官方似无 OT/CRDT，依靠 versioning。复刻 MVP 建议沿用版本化策略。
- **Vertex 的图渲染技术栈未公开**：本文按公开能力上限做了推断（Canvas + DOM overlay 混合），复刻直接用 Sigma+WebGL+DOM 即可。
- **Functions on Models 的 Sunset 路径**：官方在 2025 年后明确推荐 Function-backed Action 路径，新复刻只需实现这一条，无需走 Modeling Objectives 直接绑定的 legacy 模式。

- **Diagramming 模式**：Vertex 还有一个“Diagramming”模式（beta，P&ID-like 工程图绘制），与基于 Ontology 自动布局的 graph 模式互补；MVP 不必复刻，可放到 P3。
-

参考链接汇总

Palantir 官方文档 - Vertex

- [Vertex Configuration / Control Panel settings](#)
- [Embed a graph in a Workshop module](#)
- [Explore related time series](#)
- [Generate a graph from other applications](#)
- [Object and edge display options](#)
- [Scenarios - Getting started](#)
- [Scenario options](#)
- [Use time selection](#)
- [Explore object relationships \(Search Around\)](#)
- [Create a graph template](#)
- [Save, share, and collaborate](#)
- [Configure thresholds](#)
- [View and filter events on a timeline](#)
- [Events overview](#)
- [Configure chained models \[Sunset\]](#)
- [Example use cases](#)

Palantir 官方文档 - Workshop & Ontology

- [Workshop Scenarios - Core concepts](#)
- [Workshop Scenarios - Overview](#)
- [Workshop Scenarios - Getting started](#)
- [Workshop Scenarios - Apply scenarios](#)

- [Workshop Scenarios - Select a model](#)
- [Ontology overview](#)
- [Why create an Ontology?](#)

Palantir 官方文档 - Functions & Models

- [Functions on models](#)
- [Model functions guide](#)
- [Create model functions](#)
- [Create model Functions \(index\)](#)
- [Language models in TypeScript v1 functions](#)

Palantir 官方文档 - Branching & AIP

- [Foundry Branching - Overview](#)
- [Foundry Branching - Supported functionality](#)
- [AIP Evals - Evaluate Ontology edits](#)

官方白皮书与专利

- [Palantir Foundry for Energy \(Vertex for Oil & Gas\) PDF](#)
- [USPTO 10,990,425 - Simulated change of immutable objects during execution runtime](#)
- [USPTO 11,537,418 - Simulated change of immutable objects \(cont.\)](#)

开源工具与组件

- [Blueprint.js](#) · [GitHub](#)
- [Sigma.js](#) · [Graphology](#)
- [Cytoscape.js](#) · [G6 \(AntV\)](#) · [Cosmograph](#)
- [dagre](#) · [elkjs](#)
- [uPlot](#) · [Visx](#)
- [Temporal](#) · [Prefect](#) · [Dagster](#)
- [TimescaleDB](#) · [QuestDB](#) · [Dolt](#)
- [Triton Inference Server](#)

- [Zustand](#) · [TanStack Query](#) · [Radix UI](#) · [dnd-kit](#) · [React Hook Form](#) · [Zod](#)